

Design Patterns Review

Strategy and State Pattern

Software Re-Engineering CS4118
Lect 8

Designing a scenerio

You are designing an **online shopping system**.

The system must support multiple discount types:

- Regular Discount (5%)
- Premium Customer Discount (10%)
- Seasonal Discount (20%)
- No Discount
- We may add Festival Discount, Clearance Discount, Coupon-based Discount, and Flash Sale Discount.

Naïve Approach

```
+-----+
|      Order      |
+-----+
| - amount        |
| - customerType  |
| - season        |
+-----+
| + calculateTotal() |
+-----+
```

```
if customerType == "Regular"
    apply 5%
else if customerType == "Premium"
    apply 10%
else if season == "Seasonal"
    apply 20%
else
    no discount
```

Design Flaws in This Approach

Violates Open–Closed Principle (OCP)

Every time a new discount is introduced:

- You must modify `Order`
- Risk of breaking existing logic

`Order` is NOT closed for modification.

Design Flaws in This Approach

Low Cohesion

Order now handles:

- Order data
- Business rules
- Discount algorithms
- Too many responsibilities.

Design Flaws in This Approach

High Coupling

Order depends on:

- Customer type
- Season
- Discount rules
- Any change in discount logic affects Order.

Rethinking the Design

- What is varying in the system?
- The **discount algorithm** varies.
- Design Principle Suggests: Separate the code that varies, from code that stays same.
- So discount behavior should be separated.

Applying Strategy Pattern

- Create Strategy Interface

```
+-----+  
|  DiscountStrategy  |  
+-----+  
| + applyDiscount()  |  
+-----+
```

Applying Strategy Pattern

- Concrete Strategies

```
+-----+  
| RegularDiscount      |  
+-----+  
| + applyDiscount()    |  
+-----+
```

```
+-----+  
| PremiumDiscount      |  
+-----+  
| + applyDiscount()    |  
+-----+
```

```
+-----+  
| SeasonalDiscount     |  
+-----+  
| + applyDiscount()    |  
+-----+
```

```
+-----+  
| NoDiscount           |  
+-----+  
| + applyDiscount()    |  
+-----+
```

Applying Strategy Pattern

- Modify Order (Context) class

```
+-----+
|      Order      |
+-----+
| - amount        |
| - discountStrategy |
+-----+
| + setDiscountStrategy() |
| + calculateTotal()      |
+-----+
```

Design Improvements

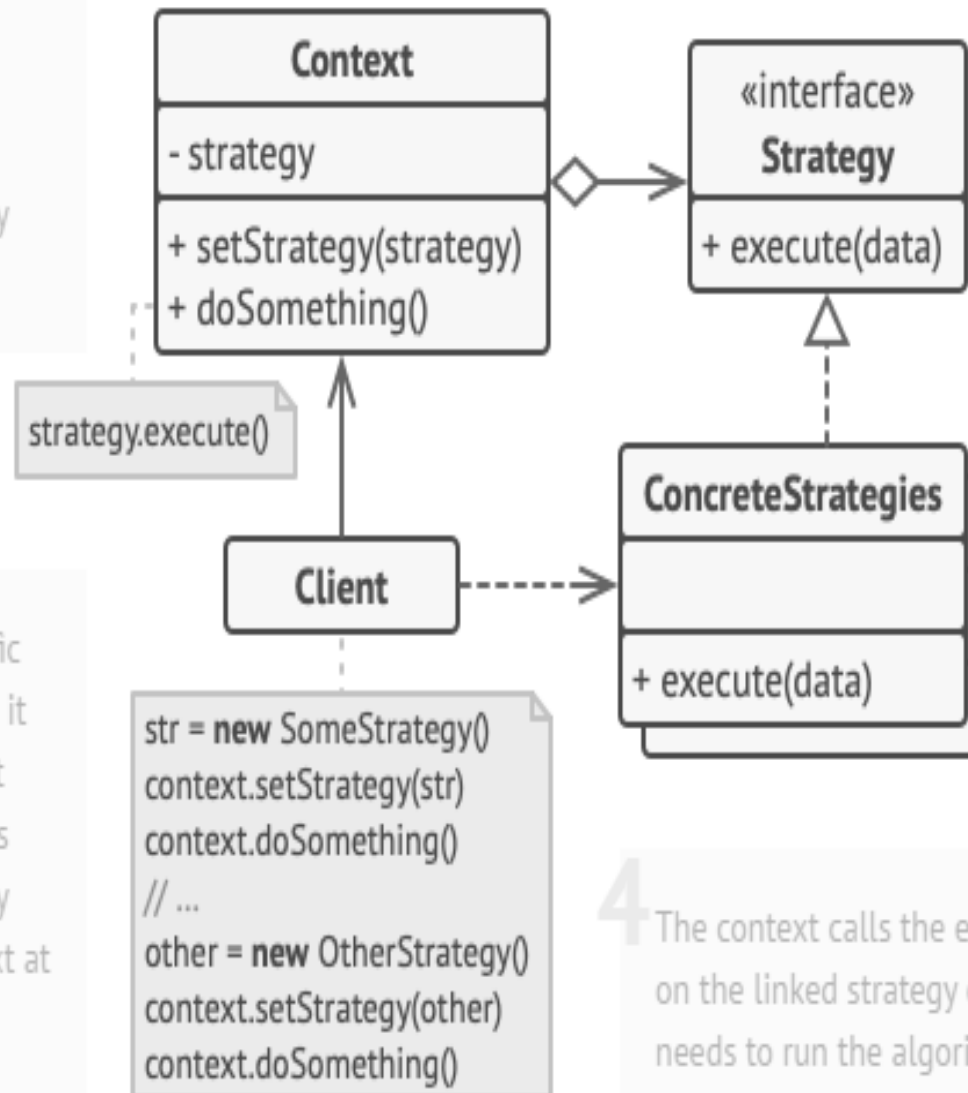
- Open–Closed Principle Achieved?
- Adding Festival Discount?
- High Cohesion?
- Order → manages order
- DiscountStrategy → manages discount logic
- Low Coupling?
- Order depends only on: Discount Strategy

Code of new Design

Can you write code of new order class along with StrategyClasses?

Strategy is a behavioral design pattern that lets you define a family of algorithms, put each of them into a separate class, and make their objects interchangeable.

1 The **Context** maintains a reference to one of the concrete strategies and communicates with this object only via the strategy interface.



2 The **Strategy** interface is common to all concrete strategies. It declares a method the context uses to execute a strategy.

3 **Concrete Strategies** implement different variations of an algorithm the context uses.

5 The **Client** creates a specific strategy object and passes it to the context. The context exposes a setter which lets clients replace the strategy associated with the context at runtime.

4 The context calls the execution method on the linked strategy object each time it needs to run the algorithm. The context doesn't know what type of strategy it works with or how the algorithm is executed.

Designing a scenerio

An order in an e-commerce system can be in different modes:

- New , Paid , Shipped , Delivered ,Cancelled
- A New order can be Paid or Cancelled
- A Paid order can be Shipped
- A Shipped order can be Delivered
- Delivered order cannot change mode

Naïve Approach

```
+-----+
|      Order      |
+-----+
| - state          |
+-----+
| + pay()          |
| + ship()         |
| + deliver()      |
| + cancel()       |
+-----+
```

```
if(state.equals("New")) {
    state = "Paid";
} else {
    throw error;
}
```

Design Flaws

Violates Open–Closed Principle

- If you add a new mode:
- Modify all methods
- Add new conditions everywhere

Poor Cohesion

- Order class handles:
- Data
- State transitions
- Business rules
- Validation logic

Applying State Pattern

```
+-----+
|  OrderState  |
+-----+
| + pay()      |
| + ship()     |
| + deliver()  |
| + cancel()   |
+-----+
```

```
+-----+
|  NewState    |
+-----+
+-----+
|  PaidState   |
+-----+
+-----+
|  ShippedState |
+-----+
+-----+
|  DeliveredState |
+-----+
```

Applying State Pattern

```
-----+
      Order      |
-----+
- state: OrderState |
-----+
+ setState()      |
+ pay()           |
+ ship()          |
+ deliver()       |
+ cancel()        |
-----+
```

```
state.pay(this);
```

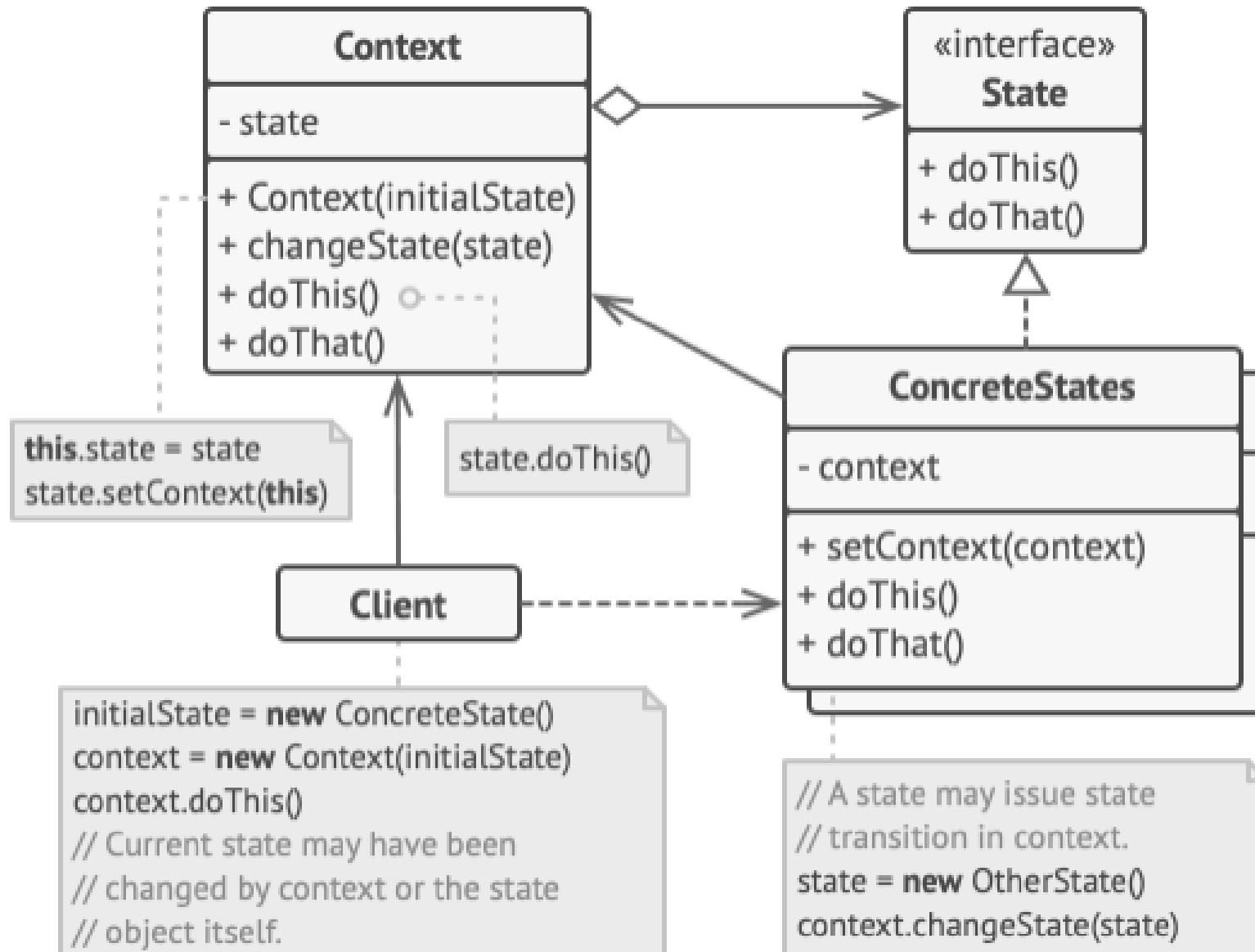
state object decides:

- Is action allowed?
- What next state will be?

State Pattern Defined

State is a behavioral design pattern that lets an object alter its behavior when its internal state changes. It appears as if the object changed its class.

Class Diagram



Benefits of State Pattern

- ✓ Eliminates conditional logic
- ✓ Encapsulates transition rules
- ✓ Makes adding new states easy
- ✓ Improves cohesion
- ✓ Makes behavior state-driven

Homework

- See how Template Method pattern works
- Its applications

References

- Gang of Four Design Patterns
- <https://refactoring.guru/design-patterns/state?authuser=0>
- <https://refactoring.guru/design-patterns/strategy?authuser=0>
- Head First Design Patterns by Eric